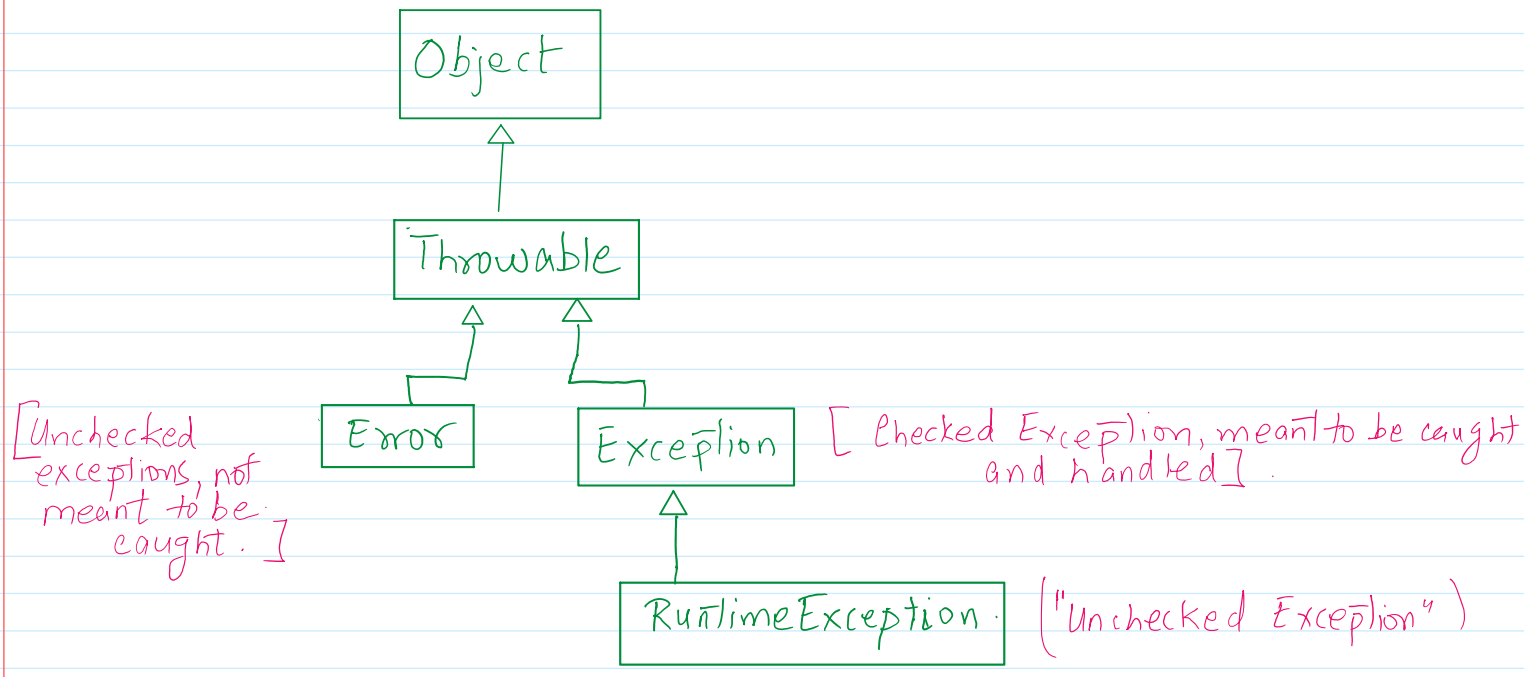
Exception Handling Part-2

23 June 2024 09:57

The **'Exception'** class in Java is the base class for all exceptions. It is a subclass of the **'Throwable'** class. And is a part of Java's exception handling mechanism, which. allows you to handle runtime errors gracefully. Understanding the **'Exception'** class is crucial for effective exception handling in Java.

Hierarchy of the 'Exception' class.



Constructors of the exception class:
The exception class has several constructors for creating exception objects:

1. **Default constructor:** Creates an **'Exception'** object with no detail message.
   public Exception()
2. **Constructor with a message:** Creates an **'Exception'** object with a detailed message.
   public Exception(String message)

3. **Constructor with a cause:** Creates an **'Exception'** object with a cause.
   public Exception(Throwable cause)
   Parent class reference, such that any child class object can be passed.

4. **Constructor with a message and a cause:** Creates an **'Exception'** object with a detailed message and cause.
   public Exception(String message, Throwable cause)

Commonly used Exception Classes

- **IOException:** Throws when an I/O operation fails or is interrupted.
- **SQLException:** Thrown when there is an error in accessing the database.
- **FileNotFoundException:** Thrown when an attempt to open the file denoted by a specified path name has failed.
- **ClassNotFoundException:** Thrown when an application tries to load a class through its

string name, but no definition for the class with a specified name could be found.

Creating Custom Exceptions

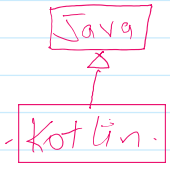
You can create your own custom exceptions by extending the exception class or any of its subclasses:

Example of Checked Exception:

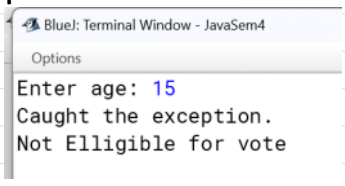
```
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class TestCustomEHExample8 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestCustomEHExample8
{
   public static void main(String args[]){
      int age;
      Scanner sc = new Scanner(System.in);
      try{
         System.out.print("Enter age: ");
         age = sc.nextInt();
         validateAge(age);
      }
      catch(CustomEHExample8 e){
         System.out.print("Caught the exception.");
         System.out.print("\n"+e.getMessage());
      }
   }

   public static void validateAge(int age) throws CustomEHExample8{
      if (age < 18){
         throw new CustomEHExample8("Not Elligible for vote");
      }
      else{
         System.out.print("\nElligible to vote.");
      }
   }
}
```



Output:

1. 

2. **No Exception is generated:**

```
BlueJ: Terminal Window - JavaSem4
Options
Enter age: 18

Elligible to vote.
```

Example of Unchecked Exception

```
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class TestCustomUncheckedException here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestCustomUncheckedException
{
   public static void main(String args[]){
      int age;
      Scanner sc = new Scanner(System.in);
      try{
         System.out.print("Enter age: ");
         age = sc.nextInt();
         validateAge(age);
      }
      catch(CustomUncheckedException e){
         System.out.print("Caught the exception.");
         System.out.print("\n"+e.getMessage());
      }
   }
   public static void validateAge(int age){
      if (age < 18){
         throw new CustomUncheckedException("Not Elligible for vote");
      }
      else{
         System.out.print("\nElligible to vote.");
      }
   }
}
```

Output:

1.

```
BlueJ: Terminal Window - JavaSem4
Options
Enter age: 17
Caught the exception.
Not Elligible for vote
```

2.

```
BlueJ: Terminal Window - JavaSem4
Options
Enter age: 20

Elligible to vote.
```

```
Try-with-resource Statement
try(FileReader r = new FileReader("file.txt")){
    //use the reader object to read the file.
}
catch(IOException e){
    e.printStackTrace();
}
```

This concept of Exception handling is done with File Handling concept.

Chained Exception

Chained exception in Java allow you to relate one exception to another, forming a chain of exceptions. This is particularly useful when an exception is caused by another underlying exception providing, a more comprehensive understanding of what went wrong. Chained exceptions are supported through constructor and methods in the **'Throwable'** class.

Why use chained exceptions?

- **Reserve original cause:** Helps in preserving the original cause of an exception when catching and throwing a new exception.
- **Enhanced debugging:** Provides more context and detail about the error, making it easier to debug.
- **Error propagation:** Allows the propagation of exceptions with detailed context through different layers of an application.

Basic Mechanism

When you catch an exception and throw another exception in response, you can chain the exception using the constructor of **'Throwable'** that takes a '**Throwable'** as an argument or using the '**intCause()'** method.

Constructors and methods for chaining exceptions

Constructors

1. **Constructor with cause parameter.**
   i) **public Throwable(String message, Throwable cause)**
   ii) **public Throwable(Throwable cause)**

   Subclasses of **'Throwable'** exception(like **'Exception'** and **'RuntimeException')** also provide these constructors.

2. **'initCause()' method:**
   Set the cause of the exception.
   public Throwable initCause(Throwable cause)

Example of chained exceptions.

Using Constructors

```
package ExceptionHandling;


/**
 * Write a description of class ChainedException here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ChainedException
```

```
{
    public static void main(String args[]){
        try{
            method1();
        }
        catch(Exception e){
            e.printStackTrace();
        }
    }

    public static void method1() throws Exception{
        try{
            method2();
        }
        catch(Exception e){
            throw new Exception("Exception in method1() ", e);
        }
    }

    public static void method2() throws Exception{
        throw new Exception("Original cause of the exception");
    }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options

Can only enter input while your program is running
java.lang.Exception: Exception in method1()
        at ExceptionHandling.ChainedException.method1(ChainedException.java:26)
        at ExceptionHandling.ChainedException.main(ChainedException.java:14)
        at ExceptionHandling.__SHELL6.run(__SHELL6.java:6)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at bluej.runtime.ExecServer$3.lambda$run$0(ExecServer.java:873)
        at bluej.runtime.ExecServer.runOnTargetThread(ExecServer.java:996)
        at bluej.runtime.ExecServer$3.run(ExecServer.java:870)
Caused by: java.lang.Exception: Original cause of the exception
        at ExceptionHandling.ChainedException.method2(ChainedException.java:31)
        at ExceptionHandling.ChainedException.method1(ChainedException.java:23)
        ... 9 more
```

Using 'initCause()' Method

```
package ExceptionHandling;


/**
 * Write a description of class ChainedExceptionInitCauseMethod here.
```

```
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ChainedExceptionInitCauseMethod
{
   public static void main(String args[]){
      try{
         method1();
      }
      catch(Exception e){
         e.printStackTrace();
      }
   }

   public static void method1() throws Exception{
      try{
         method2();
      }
      catch(Exception e){
         Exception ne =  new Exception("Exception in method1() ");
         ne.initCause(e);
         throw ne;
      }
   }

   public static void method2() throws Exception{
      throw new Exception("Original cause of the exception");
   }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options

Can only enter input while your program is running
java.lang.Exception: Exception in method1()
        at ExceptionHandling.ChainedExceptionInitCauseMethod.method1(ChainedExceptionInitCauseMethod.java:26)
        at ExceptionHandling.ChainedExceptionInitCauseMethod.main(ChainedExceptionInitCauseMethod.java:14)
        at ExceptionHandling.__SHELL7.run(__SHELL7.java:6)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at bluej.runtime.ExecServer$3.lambda$run$0(ExecServer.java:873)
        at bluej.runtime.ExecServer.runOnTargetThread(ExecServer.java:996)
        at bluej.runtime.ExecServer$3.run(ExecServer.java:870)
Caused by: java.lang.Exception: Original cause of the exception
        at ExceptionHandling.ChainedExceptionInitCauseMethod.method2(ChainedExceptionInitCauseMethod.java:33)
        at ExceptionHandling.ChainedExceptionInitCauseMethod.method1(ChainedExceptionInitCauseMethod.java:23)
        ... 9 more
```

Custom Exceptions with Chaining

```
package ExceptionHandling;


/**
 * Write a description of class CustomChain here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestCustomChain
{
   public static void main(String args[]){
      try{
         method1();
      }
      catch(MyCustomException e){
         e.printStackTrace();
      }
   }

   public static void method1() throws MyCustomException{
      try{
         method2();
      }
      catch(Exception e){
         throw new MyCustomException("Exception in method1() ", e);
      }
   }

   public static void method2() throws Exception{
      throw new Exception("Original cause of the exception");
   }
}
```

BlueJ: Terminal Window - JavaSem4

Options

Can only enter input while your program is running

```
ExceptionHandling.MyCustomException: Exception in method1()
        at ExceptionHandling.TestCustomChain.method1(TestCustomChain.java:26)
        at ExceptionHandling.TestCustomChain.main(TestCustomChain.java:14)
        at ExceptionHandling.__SHELL8.run(__SHELL8.java:6)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:568)
        at bluej.runtime.ExecServer$3.lambda$run$0(ExecServer.java:873)
        at bluej.runtime.ExecServer.runOnTargetThread(ExecServer.java:996)
        at bluej.runtime.ExecServer$3.run(ExecServer.java:870)
Caused by: java.lang.Exception: Original cause of the exception
        at ExceptionHandling.TestCustomChain.method2(TestCustomChain.java:31)
        at ExceptionHandling.TestCustomChain.method1(TestCustomChain.java:23)
        ... 9 more
```